

Data Encryption Standard

The data encryption standard (DES) defined by US NIST performs encryption in hardware thereby speeding up the encryption and decryption operation.

From: [Wireless Communications & Networking, 2007](#)

Related terms:

[Semiconductor](#), [Dielectrics](#), [Amplifier](#), [Transistor](#), [Cryptography](#), [Capacitance](#), [Amplitudes](#), [Electric Potential](#)

[View all Topics](#)

Cryptography

Rebecca N. Wright, in [Encyclopedia of Physical Science and Technology \(Third Edition\)](#), 2003

IV.C.1.a Triple-DES

Triple-DES uses two DES encryption keys, K_1 and K_2 . The triple-DES encryption C of a plaintext block M is

where *Enc* and *Dec* denote regular DES encryption and decryption. Decryption is

The reason for the encrypt/decrypt/encrypt pattern is for compatibility with regular DES: triple-DES with $K_1 = K_2$ is identical to regular DES. With independently chosen keys, triple-DES has an effective key length of 128 bits.

[> Read full chapter](#)

Security in Wireless Systems

Vijay K. Garg, in [Wireless Communications & Networking](#), 2007

Secret Key (symmetric encryption)

Both sender and recipient share a knowledge of the same secret key. The scrambling technique is called encryption. The message is referred to as plaintext or clear text, and the encrypted version of it is called ciphertext. The encryption of a plaintext x into a ciphertext y using a secret key e_k is given as (see Figure 13.10):

Figure 13.10. Encryption using secret key.

The corresponding decryption yields

where: d_k is the decryption key

Ideally, the encryption scheme should be such that it cannot be broken at all. Because there are no practical methods of achieving such an unconditional security, encryption schemes are designed to be computationally secure. The encryption and decryption algorithms use the same key, and, hence, such algorithms are called *symmetric key algorithms*. The symmetric key algorithm is vulnerable to interception and key management is a challenge. The strength of this algorithm depends upon length of key. Longer keys are more difficult to break. If the length of a secret key is n bits, at least 2^{n-1} steps would be required to break the encryption.

The data encryption standard (DES) defined by US NIST performs encryption in hardware thereby speeding up the encryption and decryption operation. Additional features of DES are:

1. DES is a block cipher and works on a fixed-size block of data. The message is segmented into blocks of plaintext, each comprising 64 bits. A unique 56-bit key is used to encrypt each block of plaintext into a 64-bit block of ciphertext. The receiver uses the same key to perform the decryption operation on each 64-bit data block it receives, thereby reassembling the blocks into a complete message.
2. The larger the key, the more difficult it is for someone to decipher it. DES uses a 56-bit key and provides sufficient security for most commercial applications. Triple-DES is the extended version of DES which applies DES three times with two 56-bit keys.

International data encryption algorithm (IDEA) is a block cipher method similar to DES. It operates on 64-bit blocks of plaintext and uses a 128-bit key. The algorithm can be implemented either in hardware or software. It is three times faster than DES and is considered superior to DES.

The key sizes used in current wireless systems are not sufficiently large enough for good security. IS-136 uses a 64-bit A-key that is secure, but is still considered to be weak.

[> Read full chapter](#)

Design Optimization Example

Peter Wilson, in [Design Recipes for FPGAs \(Second Edition\)](#), 2016

19.2 The Data Encryption Standard

The Data Encryption Standard, usually referred to by the acronym DES, is a well-established encryption algorithm which was first standardized by NIST in the 1980s. It is described in detail earlier in this book, in Chapter 10, so only the basic information about the algorithm is presented here.

While DES has largely been superseded by the AES (Advanced Encryption Algorithm) it is now common to find the algorithm being used in triplicate (an algorithm known as Triple-DES or TDES for short). This algorithm uses the same DES core, but uses three passes with different keys. DES was designed to be small and fast, and the algorithm is mainly based on shuffling and substitution. There is very little computation involved, which makes it ideal for hardware implementation.

[> Read full chapter](#)

Secure Systems

Peter Wilson, in [Design Recipes for FPGAs \(Second Edition\)](#), 2016

10.1 Introduction to Block Ciphers

The data encryption standard (DES) is a symmetric block cipher. A stream cipher operates on a digital data stream one or more bits at a time. A block cipher operates on complete blocks of data at any one time and produces a ciphertext block of equal size. DES is a block cipher that operates on data blocks of 64 bits in size. DES uses a

64-bit key 8×8 including 1 bit for parity, so the actual key is 56 bits. DES, in common with other [block ciphers](#), is based around a structure called a *Feistel Lattice* so it is useful to describe how this works.

[> Read full chapter](#)

Security Protocols and Algorithms

Timothy Stapko, in [Practical Embedded Security](#), 2008

DES

The [Data Encryption Standard](#) was developed in the early 1970s by a team from IBM with some help from the National Security Association (NSA). Many of the details of the implementation remained secret for many years, as the NSA did not want to reveal what techniques it had up its sleeves. The algorithm IBM had created was originally called Lucifer and was modified and tweaked by the IBM team and the NSA as a response to a call from the NIST precursor for a government standard [encryption](#) scheme. The public response was enthusiastic but underwhelming (no one was really studying [cryptography](#) at the time), so the design from the IBM team was a shoe-in. DES was originally intended to be implemented in hardware and was designed around the technology of the time. However, the published specification contained enough information for software implementations to be designed once general-purpose computers became more accessible. DES by itself is a fairly quick algorithm, but its standard 56-bit key is too small to be practical anymore (a 56-bit key can be broken using brute force methods over a distributed network in very little time). Triple-DES, or 3DES, uses 3 56-bit keys to achieve a higher level of security, and is more standard today. Despite its age and suspicious involvement of the NSA in its design, DES has stood the test of time and still remains a viable cryptographic algorithm.

The implementation of DES has some small challenges in software, as a couple of operations are trivial to implement in hardware but require some additional work to implement programmatically. The math behind DES is pretty heavy-duty (you can read all about it in [Applied Cryptography](#)), but the implementation is pretty straightforward, since most of the tricky math is actually wrapped up in a collection of constants that are used essentially to randomize the data being encrypted. These constants are put into what are called *substitution boxes*, or s-boxes, and *permutation boxes* (p-boxes). The real security of DES resides in the s-boxes and in the key—we aren't going to worry about why DES is secure (plenty of books and papers have been written on the subject)—for our purposes we can just assume that it is. We will

instead look at the implementation and the tradeoffs we can exploit to make DES work for an embedded application.

One of the advantages of DES is that it was designed in a time when all hardware resources were scarce. The algorithm predates 32-bit and even 16-bit buses and is specifically designed to function in an 8-bit world. This property dates the algorithm, but it is also a boon to modern embedded developers, since the operations are designed for 8-bit computing. DES is also designed to be implemented directly in hardware, which can be a big advantage for performance. Conceivably, it would be very easy to add a DES coprocessor to an embedded design and not add too much to the cost.

Unfortunately, the use of DES is discouraged by pretty much everyone for new development and designs, so its utility is a bit limited. In fact, original DES should not be used at all, since it is considered too weak (see the explanation earlier in this chapter), so 3DES is really the only option. Despite the fact you shouldn't use DES anymore, it is still implemented in a wide array of applications, and if you are attempting to interface with a legacy product, it may require DES, so we include it for this reason.

[> Read full chapter](#)

General Cryptographic Concepts

Ido Dubrawsky, in [Eleventh Hour Security+](#), 2010

DES

Standing for Data Encryption Standard, this is another of those generic names that indicates it comes out of NIST. The DES algorithm uses a 56-bit key, and as expected from something with such a small key size, it is a symmetric key encryption algorithm—asymmetric keys are usually more than a thousand bits in length. It is also a “block” encryption algorithm, meaning that it encrypts in blocks—in DES's case, a block is 64 bits—one block at a time. While block ciphers are not in themselves designed to encrypt streams of data, it is possible to use what is known as a “mode of operation” to encrypt a stream using the block cipher.

[> Read full chapter](#)

Encryption

9.3.3 1997 (2001): Advanced Encryption System (AES)

However, the continually increasing speed of computers and users' ability to crack the DES code convinced the NIST that Triple DES would only be a temporary solution and on January 2, 1997, even before Triple DES was in place as the U.S. encryption standard, NIST announced plans for an advanced encryption system (AES). Later that same year, on September 12, 1997, the NIST issued a formal RFP for designs for a more secure encryption algorithm. The RFP stated that the selected proposal must be for a private-key, block-cipher system using a minimum 128-bit key, with upward capability to 192-bit and 256-bit keys. It must also be publicly disclosed, available worldwide, and distributed royalty-free. AES is expected to be completed by the summer of 2001 and to become the new federal standard, replacing Triple DES, after that time.

[> Read full chapter](#)

Quality of service and security

Dimitrios Serpanos, Tilman Wolf, in [Architecture of Network Systems](#), 2011

Symmetric key cryptography

Symmetric ciphers use the same secret key for the encryption and decryption of information. Parties that want to use such a cipher need to agree on the same secret key before using the cipher. For example, the secret can be shared by the parties when they meet in person, or the key can be exchanged using a different security protocol (e.g., using a trusted third party).

A very simple example of symmetric key cryptography is shown in Figure 10-8. The sender shown on the left uses the encryption algorithm to convert the cleartext information into ciphertext (shaded). The encrypted information is sent across the network. On the receiver side on the right, the decryption algorithm uses the same secret key to decrypt the ciphertext back into plaintext. This illustration does not show practical considerations for secure network protocols (e.g., packets need a cleartext IP header to be handled by the network).

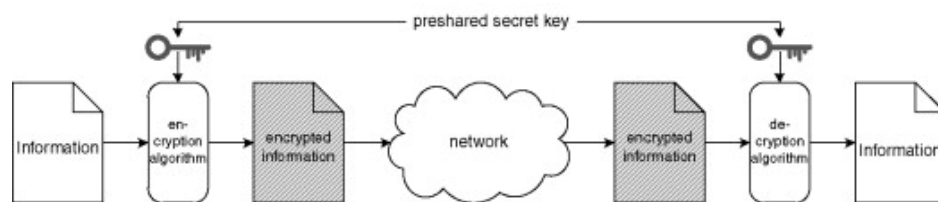


Figure 10-8. Symmetric key cryptography in network setting.

A very important aspect of this (and any other) encryption and decryption process is that the algorithm used for the process is not secret—only the keys are secret. In fact, most practically used symmetric key algorithms are standardized and well documented. Thus, anyone can program the same encryption and decryption algorithm. This approach to limiting secrecy to key material only is an important principle of security engineering. Kerckhoffs' principle summarizes this idea by stating that a cryptosystem should be secure even if everything about the system, with exception of the secret key, is publicly known. In contrast, developing an encryption/decryption process that relies on secrecy for the algorithm has several disadvantages. First, if any information about the algorithm is leaked, all instances of the system become vulnerable. Second, the lack of public scrutiny of the algorithm in the design phase makes it more likely that there are fundamental flaws in the system not detected by the designers.

Two symmetric key encryption algorithms are widely used in networks. These are:

- Data Encryption Standard (DES): The DES encryption algorithm is a “block cipher” that encrypts information in blocks of 64 bits (8 bytes). Using a 56-bit key, DES encrypts each block in 16 identical rounds. Each round operates on alternating 32-bit halves of the block and uses different 48-bit subkeys derived from the original key. The main operation in each round is the substitution of input bits through 8 S-boxes, which perform a nonlinear transformation based on a lookup table. The final step of each round is a permutation of the outputs of the S-boxes. The design of DES allows the same sequence of operations to be used for encryption and decryption. The only difference is that the sequence of subkeys needs to be reversed. DES was published as a U.S. federal standard in 1977. Triple DES, an improvement to DES, was published in 1999. Triple DES uses three encryption steps with different keys to improve the better security. More details on the functionality of these algorithms can be found elsewhere [130].
- Advanced Encryption Standard (AES): The AES encryption algorithm is also a block cipher with 128-bit blocks. The block is arranged in a 4×4 grid of bytes. There are three variants with key sizes of 128, 192, and 256 bits. Larger key sizes increase the security of the algorithm but require more processing: 128-bit keys use 10 rounds, 192-bit keys use 12 rounds, and 256-bit keys use 14 rounds. In each round, each byte is substituted with another based

on a lookup table. Then, the columns of the 4×4 grid are shifted cyclically. Next, each column is multiplied with a fixed polynomial. Finally, each byte is combined with the key used in that round. AES was published as a U.S. federal standard in 2001. The algorithm was chosen among numerous competitors through an extensive selection process. One of the concerns when selecting the algorithm was to ensure that it can be implemented efficiently in software and in hardware. The latter allows for high-throughput encryption in systems that need to support high data rates (e.g., network devices). The details of AES are described elsewhere [131].

Symmetric key encryption algorithms are widely used to encrypt data traffic in networks. However, their main limitation lies in the need for a preshared key. In a large-scale network it is impractical to assume that all pairs of end systems have a secret key setup. Therefore, it is necessary to have an additional system for the exchange of keys that can be used by symmetric key algorithms. Asymmetric key cryptography can provide such a key exchange mechanism (as well as other security functions).

[> Read full chapter](#)

Fusing DNA with digital immunity ecosystem

Dr.Rocky Termanini, in [Storing Digital Binary Data in Cellular DNA](#), 2020

The encryption algorithm

The following is the sophisticated encryption algorithm method that uses DNA coding to increase encryption strength, as shown in Fig. 10.17:

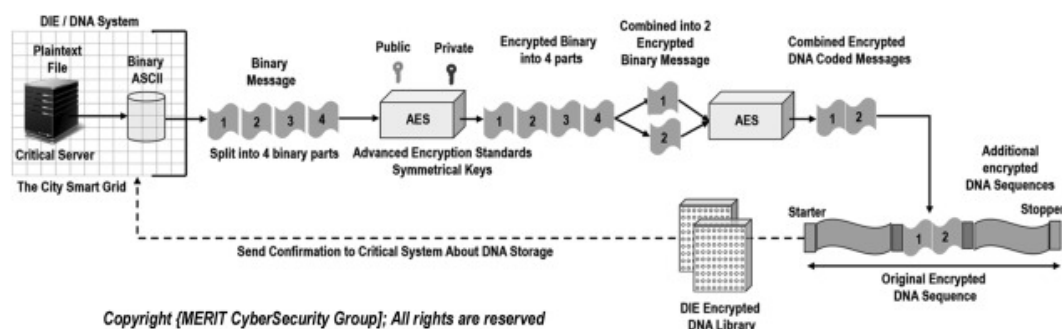


Figure 10.17. This diagram represents the advantage of storing the critical system operation data on DNA in encrypted format, with two rounds of encryption and storing the file in separate blocks. Furthermore, some “dummy DNA” sequences

are tagged to the original DNA file. It makes hacking extremely challenging, if not impossible. Copyright [MERIT CyberSecurity Group]; All rights are reserved.

Step 1: We select the binary file of the critical system to be encrypted. The binary is in ASCII decimal format (0 and 1).

Step 2: The binary data are grouped into four blocks and encrypted using a traditional Data Encryption Standard (DES), which is known as the symmetric key algorithm.

Step 3: Then, we convert the encrypted file to binary format.

Step 4: Then, we group the binary-encrypted file group into two blocks and convert it into DNA code: as A for 00, T for 01, G for 10, and C for 11.

Step 5: We then add primer/stopper on either side of this message. Primers will act as stoppers and detectors for the message. This must be done before sending the encrypted message across the grid.

Step 6: Then, we add to the original encrypted file an additional DNA sequence prepared by the sending encrypted followed by another primer/stopper.

Step 7: We add several additional DNA sequences on both sides of the original encrypted file, or we confine the message to a microdot in the microarray (secure holding place).

Step 8: The final compounded sequence is sent to the destination across the grid.

DNA data storage (DDS) is already encrypted as a jumble of ACTG; however, hackers can use sequencers to convert the coded messages to plaintext. Data encryption becomes necessary to protect the DIE biodata.

Here is another efficient approach to implementing cryptography in DNA, for the tightest security and impossible to crack: For cryptographic purposes, the interest is to generate very long one-time pad (OTP)—see (Appendix A)—as cryptographic keys, which ensures the unreachability of a cryptosystem, to convert the classical cryptographic algorithms to DNA format to benefit the advantages that DNA offer and to find new algorithms for biomolecular computation (BMC), as DNA XOR is an XOR gate that implements the exclusive OR.

[> Read full chapter](#)

A Tutorial on Software Obfuscation

Sebastian Banescu, Alexander Pretschner, in [Advances in Computers](#), 2018

4.1.4 White-Box Cryptography

This transformation was pioneered by Chow et al. [37, 38], who proposed the first white-box data encryption standard (WB-DES) and white-box advanced encryption standard (WB-AES) ciphers in 2002. The goal of white-box cryptography (WBC) is the secure storage of secret keys (used by cryptographic ciphers), in software, without hardware keys or trusted entities. Instead of storing the secret key of a cryptographic cipher separately from the actual cipher logic, WBC embeds the key inside the cipher logic. For instance, for Advanced Encryption Standard (AES) ciphers, the key can be embedded by multiplication with the T-boxes of each encryption round [39]. However, simply embedding the key in the T-boxes of AES is prone to key extraction attacks since the specification of AES is publicly known. Therefore, WB-AES implementations use complex techniques to prevent key extraction attacks, e.g., wide linear encodings [40], perturbations to the cipher equations [41], and dual-ciphers [42].

Listing 4

8-Bit XOR With MSB of Secret Key

```
1 char xor (char input) { 2 return input ^ 0x53; 3 }
```

Listing 5

LUT-Based 8-Bit XOR

```
1 char lut [256] = { 2 0x53, 0x52, 0x51, ..., 0x5C, 3 0x43, 0x42, 0x41, ..., 0x4C, 4 -
0x73, 0x72, 0x71, ..., 0x7C, 5 | 6 0xA3, 0xA2, 0xA1, ..., 0xAC 7 }; 8 9 char xor
(char input) {10 return lut [input];11 }
```

The idea behind the white-box approach in [38] is to encode the internal AES cipher logic (functions) inside look-up tables (LUTs). One extreme and impractical instance of this idea is to encode all plaintext–ciphertext pairs corresponding to an AES cipher with a 128-bit key, as a LUT with 2^{128} entries, where each entry consists of 128-bits. Such a LUT would leak no information about the secret-key but exceed the storage capacity of currently available devices. However, this LUT-based approach also works for transforming internal AES functions (e.g., XOR functions, AddRoundKey, SubBytes, and MixColumns [39]) to table lookups, which can be divided such that they have a smaller input and output size. For instance, Listing 4 shows the implementation of a 8-bit XOR gate, which takes one byte value as an input argument and outputs the bitwise-XOR of this value and the most significant byte (MSB) of the secret key of the AES cipher instance, which is 0x53 in Listing 4. This function can easily be converted to a LUT-based implementation—as illustrated in Listing 5—by constructing a LUT containing all input-output combinations for the 8-bit XOR function from Listing 4. Therefore, this LUT contains 256 elements

and the LUT-based version of the XOR function simply requires a look-up in this table, as shown on line 10 of Listing 5.

LUTs can also be used to encode random invertible bijective functions, which are used to further obfuscate the LUTs representing internal AES functions. This is necessary because an attacker could extract the MSB of the secret key from the LUT in Listing 5. In order to hide the key, WBC proposes to generate a random permutation of 256 bytes and apply it to the LUT. To be able to use the resulting LUT, the inverse permutation would be composed with the next operation following the XOR during AES encryption. Since the attacker would not know the randomly generated permutation value, she/he would no longer be able to extract the key directly from the LUT. Converting several steps of an AES cipher to LUT-based implementations and then applying random permutations to these LUTs leads to an implementation which is much more compact than the huge LUT with 2^{128} entries of 128-bits. Typically, the size of a WB-AES cipher is around a few megabytes. The same idea can also be applied to other ciphers as well.

Attacks: Several MATE attacks on WB-DES [5, 43] and WB-AES [44, 45] have been published in the literature, on the ground of algebraic attacks, which treat each cipher as an overdefined system of equations [46]. All of these attacks assume that the structure and purpose of the LUTs is known. Therefore, Banescu et al. [47] propose using data obfuscation techniques to hide the location and structure of the LUTs used by WBC ciphers.

[> Read full chapter](#)